

ON-GA

가족과 함께하는 편안한 여행 챗봇

개발 고찰 보고서

공식 관광데이터 기반 RAG · Rule Engine · 가족 편의정보 서비스

SSAFY EnjoyTrip ChatBot Project

2026. 09.

1. 프로젝트 개요

ON-GA는 부모님, 영유아, 또는 온 가족이 함께 여행할 때 필요한 이동·편의시설 정보를 공식 관광데이터 기반으로 제공하는 전국 단위 여행 챗봇이다. 일반적인 관광지 추천보다 “우리 가족이 이 장소를 이용할 때 어떤 점을 확인해야 하는가”에 초점을 맞췄다.

한국관광공사의 무장애 여행 정보를 기반으로 장소를 검색하고, 계단·경사로·엘리베이터·화장실·수유실·유모차·휠체어·휴식공간 등 가족 여행에서 실제 의사결정에 필요한 항목을 구조화하였다. 이후 사용자가 선택한 조건과 공식 데이터를 Rule Engine으로 비교하고, LLM은 해당 결과를 자연어로 설명하는 역할을 담당하도록 설계하였다.

주요 담당 및 구현 범위

- 한국관광공사 TourAPI 연동 및 전국 관광지 검색
- 관광지명 추출, 동명이인·모호 장소 식별 및 후보 재정렬
- 무장애 여행 정보 정규화와 false/null 의미 분리
- 가족 조건 기반 Comfort Rule Engine 설계
- 시설 정보 구조화 및 map_search_text 생성
- LangGraph·ChromaDB 기반 RAG 흐름과 LLM 응답 보조
- current_place_id를 이용한 후속 질문 문맥 유지
- Front-End와 API Contract 통합 및 실제 데이터 QA

핵심 설계 원칙

“공식 데이터가 말하지 않은 내용은 서비스도 단정하지 않는다.”

이 원칙은 프로젝트 전반의 판단 기준이 되었다. 특히 가족 여행 정보는 잘못된 안내가 실제 이동 계획에 영향을 줄 수 있으므로, 생성형 AI의 자연스러운 답변보다 데이터의 신뢰성과 불확실성 표현을 우선하였다.

2. 설계 과정에서의 주요 고민

2.1 LLM과 Rule Engine의 역할 분리

초기에는 외부 관광 데이터를 검색한 뒤 LLM이 상황을 종합하여 답변하도록 구성하면 충분하다고 생각했다. 그러나 실제 데이터를 확인하면서 “유모차 대여 가능”과 “유모차로 전체 동선을 이동하기 편함”, “장애인 주차장 있음”과 “일반 고령자가 이용 가능함”은 서로 다른 사실이라는 점이 드러났다. LLM이 이러한 정보를 자연스럽게 연결하면서 과도한 추론을 할 가능성이 있었다.

이에 핵심 판정은 코드 기반 Rule Engine 이 담당하고, LLM 은 이미 결정된 구조화 결과를 설명하는 역할로 제한하였다. 즉, 공식 데이터 → 정규화 → Rule Engine → LLM 설명 순으로 책임을 분리하였다. 이를 통해 답변의 자연스러움은 유지하면서도 핵심 판정은 결정적으로 관리할 수 있었다.

2.2 “없음”과 “확인되지 않음”의 구분

공식 데이터에 특정 시설 정보가 존재하지 않는다고 해서 해당 시설이 실제로 없다고 단정할 수는 없다. 따라서 false 는 “공식적으로 없음이 확인됨”, null 은 “공식 자료만으로 확인할 수 없음”으로 구분하였다. 사용자 화면에서도 null 을 “없음”이 아니라 “공식 자료에서 확인되지 않았어요”로 표현하였다.

이 구분은 단순한 데이터 모델링을 넘어 서비스 신뢰성에 직접 영향을 주었다. 사용자는 “없음”과 “모름”을 다르게 해석하며, 여행 계획에서는 이 차이가 실제 행동을 바꿀 수 있기 때문이다.

3. 개발 과정에서 발생한 문제와 해결

3.1 관광지 검색 정확도 문제

초기 장소 검색에서는 사용자가 “불국사”를 입력했을 때 경주 감은사지, 계림, 교촌마을처럼 직접 관련성이 낮은 후보가 포함되거나, “해운대” 검색에서 부산의 호텔·시장·공원 등 넓은 지역 결과가 함께 노출되는 문제가 발생했다. 외부 API 검색 결과를 그대로 후보로 사용하고, 질문에서 장소명을 정확히 추출하거나 후보를 서비스 목적에 맞게 재정렬하는 과정이 부족했던 것이 원인이었다.

해결을 위해 장소명 추출 로직을 개선하고, 제목 일치도와 지역 정보를 함께 반영하는 후보 ranking 을 추가했다. 관광지명과 직접 관련 없는 결과는 제거하고, 해운대처럼 실제로 여러 관광지를 가리킬 수 있는 표현은 임의로 하나를 선택하지 않고 AMBIGUOUS_PLACE 로 처리하였다. Front 에서는 candidates[]를 버튼으로 노출하고, 선택된 content_id 를 current_place_id 로 다시 전달하도록 구성했다.

이 과정에서 외부 API 의 검색 결과를 “정답”으로 사용하는 것이 아니라, 서비스 목적에 맞는 검색 후처리와 후보 품질 관리가 필요하다는 점을 확인했다.

3.2 걷기 부담(walking_difficult) 판정

요구사항에는 “많이 걸어야 하는지 알려줘”라는 질문이 있었지만 TourAPI 에는 걷기 난이도를 직접 제공하는 필드가 없었다. 예를 들어 성산일출봉에는 “관광지까지의 거리 약 100m”라는 공식 정보가 존재하지만, 이 수치만으로 부모님에게 편한지 힘든지를 판단할 수는 없다. 전체 동선, 계단, 경사도, 노면, 휴식공간 등의 정보가 함께 필요하기 때문이다.

따라서 수치가 존재한다는 이유만으로 임의 기준을 만들지 않고, 명확한 보행 부담 근거가 부족하면 UNKNOWN 으로 유지하였다. 불국사의 장애인 주차구역이나 장애인 탑승차량 정보 역시 일반 고령자에게 그대로 적용할 수 없으므로 walking_difficult 를 SATISFIED 로 판단하지 않도록 정책을 수정하였다.

이 사례를 통해 “데이터가 존재한다”와 “판단에 충분한 데이터가 존재한다”는 서로 다른 문제임을 배웠다.

3.3 Mock 데이터가 실제 서버 응답에 섞인 문제

테스트 중 성산일출봉을 조회했을 때 실제 content_id 가 아닌 Mock Fixture 의 content_id 와 “테스트로 3” 같은 주소가 반환되는 문제가 발생했다. 원인을 추적한 결과 pytest 에서 생성한 Mock cache 와 실제 TourAPI 서버의 cache 가 동일한 디렉터리를 공유하고 있었다.

테스트가 먼저 실행되면 Mock 데이터가 cache 에 저장되고, 이후 실제 서버가 해당 cache 를 우선 조회하면서 Mock 응답을 재사용한 것이다. 이를 해결하기 위해 data/cache/real 과 data/cache/mock 을 물리적으로 분리하였다. 이후 테스트 환경이 생성한 상태가 실제 실행 환경에 영향을 주지 않도록 구조를 변경했다.

테스트 환경과 운영 환경은 코드 분기만으로 분리하는 것이 아니라 캐시·저장소·환경변수 등 “상태를 공유하는 요소”까지 함께 분리해야 한다는 점을 경험했다.

3.4 LLM Hallucination 과 후처리 Guard

성산일출봉 공식 데이터에는 장애인 주차 정보만 존재했지만 LLM 이 “장애인 탑승차량 정보도 확인된다”고 답한 사례가 있었다. 처음에는 Prompt 에 “자료에 없는 내용을 말하지 말라”는 규칙을 강화했지만, 실제 응답에서는 완전히 차단되지 않았다.

이에 Prompt 만 신뢰하지 않고 구조화된 공식 데이터와 최종 answer 를 비교하는 Backend Guard 를 추가하였다. 제한시설에 대한 언급은 해당 장소의 정규화 데이터에 실제 근거가 있을 때만 허용하고, 없는 시설명은 제거하도록 변경했다. 또한 최종 사용자용 answer 는 구조화 데이터에서 1~2 문장으로 결정적으로 생성하여 내부 enum 이나 Rule Engine 원문이 노출되지 않도록 했다.

중요한 비즈니스 규칙과 사실 검증은 Prompt Engineering 만으로 보장할 수 없으며, 코드 수준의 검증 계층이 필요하다는 것을 확인했다.

3.5 Front-End / Back-End Contract 불일치

Front와 Backend를 각각 개발한 뒤 통합하는 과정에서도 예상보다 많은 세부 차이가 발견되었다. Front에서는 long_walk_difficult, UNSATISFIED, driving 등의 값을 사용하고 있었지만 Backend는 walking_difficult, CONFLICT를 사용하고 driving을 지원하지 않았다. 화면상으로는 비슷해 보여도 그대로 연결하면 422 오류 또는 잘못된 상태 표시가 발생할 수 있는 구조였다.

통합 단계에서 Request/Response schema, status enum, current_place_id, candidates, suggested_questions를 기준으로 계약을 다시 맞추고 contract-check를 추가하였다. 이후 Front가 보내는 값과 Backend가 허용하는 값을 자동 검사하여 필수 위반 0건을 유지하도록 했다.

3.6 “많이 보여주는 것”이 좋은 UX는 아니었다

Backend가 공식 데이터를 상세하게 구조화하면서 초기 Front는 가능한 정보를 많이 보여주는 방식으로 구현되었다. 그 결과 사용자 화면에 Rule Engine 결과, CHECK_NEEDED, UNKNOWN, 조건별 내부 키와 장문의 설명이 함께 노출되었다. 기능적으로는 정확했지만 일반 사용자가 원하는 것은 내부 판정 과정이 아니라 “우리 가족이 무엇을 확인해야 하는가”라는 결론이었다.

최종 통합 버전에서는 성공 응답에서 raw answer를 직접 렌더링하지 않고, 구조화 데이터를 이용해 핵심 1~2문장만 생성하도록 변경했다. 화면의 정보 순서도 판정 한 줄 → 장소 → 가족 조건 → 시설 → 접힌 상세정보 → 후속 질문으로 정리하였다. 이를 통해 개발자가 가진 정보량과 사용자가 실제로 필요로 하는 정보량은 다르다는 점을 체감했다.

문제 해결 요약

문제	원인	해결 방향
장소 검색 오탐	API 결과를 그대로 후보로 사용	장소명 추출 + 관련도 ranking + 모호성 처리
UNKNOWN 오판정	정보 없음과 시설 없음 혼동	false/null 의미 분리
걷기 부담 과대판정	거리·장애인 시설을 일반화	명확한 근거 없으면 UNKNOWN
Mock 오염	Mock/Real cache 공유	cache 디렉터리 물리적 분리
LLM Hallucination	Prompt만으로 사실성 제어	정규화 데이터 기반 Backend Guard
Front 계약 불일치	Need/status 이름 차이	Shared Contract 및 자동 검사
정보 과다 노출	Backend 정보를 그대로 출력	구조화 카드 + 1~2 문장 요약

4. 개발 결과와 잘된 점

- 한국관광공사 공식 데이터 중심으로 정보를 구성해 LLM 자체 지식 의존도를 낮췄다.
- 모르는 정보는 UNKNOWN으로 유지하여 없는 정보를 만들어내지 않도록 했다.

- Comfort, sections, facilities, sources 를 분리해 Front 가 목적에 맞게 재구성할 수 있도록 했다.
- current_place_id 를 사용하여 장소명을 반복 입력하지 않아도 후속 질문이 가능하도록 했다.
- 특정 지역 하드코딩이 아니라 전국 TourAPI 검색 구조로 구현했다.
- Front/Backend Contract 테스트와 Backend 회귀 테스트를 통해 통합 이후 오류를 줄였다.
- 사용자 화면에서는 내부 판정 용어를 숨기고 가족 조건과 시설 중심으로 재구성했다.

5. 한계 및 향후 개선 방향

5.1 공식 데이터 커버리지의 한계

무장애 여행 정보는 관광지별로 제공되는 필드의 양과 상세도가 다르다. 어떤 관광지는 엘리베이터·화장실·수유실이 잘 정리되어 있지만, 다른 관광지는 계단·휴식공간·유모차 접근 등의 정보가 비어 있다. 따라서 서비스 구조가 정확하더라도 UNKNOWN 이 많이 발생할 수밖에 없다.

5.2 걷기 부담 평가의 한계

현재 walking_difficult 는 공식 문장에 명확한 근거가 있을 때만 제한적으로 판단한다. 향후 고도, 경사도, 계단 수, 이동 거리, 노면 상태 등 객관적인 지표를 확보한다면 고령자나 보행보조기 이용자를 위한 더 세밀한 판단이 가능할 것이다.

5.3 주변 편의시설 데이터 확장

현재 시설 정보는 주로 관광지 자체에 등록된 무장애 편의시설을 사용한다. 이후 전국공중화장실표준데이터, 주차장 공공데이터 등을 결합하고 좌표 기반 반경 검색을 도입하면 “관광지 주변에서 가장 가까운 화장실·주차장”까지 확장할 수 있다.

5.4 LLM 비결정성과 평가

Rule Engine 은 동일 입력에 동일한 판정을 주지만 LLM 문장은 실행마다 조금씩 달라질 수 있다. 현재는 핵심 answer 를 구조화 데이터 기반으로 짧게 고정하여 위험을 낮췄지만, 향후 RAGAS 와 실제 사용자 평가를 함께 운영해 정확성·관련성·가독성을 지속적으로 검증할 필요가 있다.

6. 개인 고찰

이번 프로젝트를 통해 AI 서비스를 구현할 때 중요한 것은 모델 자체보다 데이터의 신뢰도와 판단 구조라는 점을 가장 크게 배웠다. 처음에는 LLM 에게 많은 역할을 맡길수록 더 “AI 다운” 서비스가 될 것이라고 생각했지만, 실제 구현에서는 오히려 핵심 사실과 판정을 코드가 통제하고 LLM 은 설명을 담당하도록 역할을 제한했을 때 서비스가 안정적이었다.

또한 기능이 많고 데이터가 자세하다고 해서 사용자 경험이 자연스럽게 좋아지는 것은 아니었다. 개발 과정 후반에는 정확한 데이터를 모두 보여주는 화면이 오히려 사용자를 피로하게 만든다는 점을 확인했고, 최종적으로 핵심 요약과 가족 조건 카드, 시설 정보 중심으로 화면을 재구성했다. 이를 통해 “정확한 시스템”과 “사용하기 좋은 서비스”는 별도로 검증해야 한다는 것을 배웠다.

Front 와 Backend 통합 과정에서도 API Contract 의 중요성을 체감했다. 각 파트가 개별적으로 정상 동작하더라도 enum, field name, 후속 질문 상태관리와 같은 작은 차이가 실제 통합에서는 큰 오류로 이어질 수 있었다.

이후 계약을 문서와 자동 테스트로 고정하면서 협업에서 인터페이스 정의가 구현만큼 중요하다는 점을 이해하게 되었다.

무엇보다 여러 차례의 실제 데이터 테스트에서 문제를 발견하고, 원인을 추적하고, 수정 후 다시 검증하는 과정을 반복하면서 “처음부터 완벽한 설계”보다 빠르게 검증 가능한 구조와 보수적인 판단 기준이 더 중요하다는 것을 경험했다.

7. 결론

ON-GA는 단순히 관광지 정보를 검색해 주는 챗봇이 아니라, 공식 데이터의 한계와 불확실성을 그대로 관리하면서 가족 여행 상황에 필요한 정보를 정리해 주는 서비스를 목표로 개발하였다. 구현 과정에서는 검색 정확도, 데이터 정규화, UNKNOWN 처리, Mock/Real 환경 분리, LLM Hallucination, Front/Backend 계약, 정보 과다 노출 등 실제 서비스 개발에서 발생할 수 있는 다양한 문제를 경험했다.

최종적으로 “공식 데이터가 말하지 않은 내용은 단정하지 않는다”, “핵심 판정은 코드가 담당하고 LLM은 설명했다”, “사용자에게 필요한 정보만 우선 보여준다”는 세 가지 원칙을 정립할 수 있었다. 향후 보조 공공데이터와 위치 기반 기능을 추가한다면 더욱 실용적인 가족 여행 지원 서비스로 확장할 수 있을 것으로 판단한다.